

Сетевое программирование на C++ с Windows API

Системное программирование

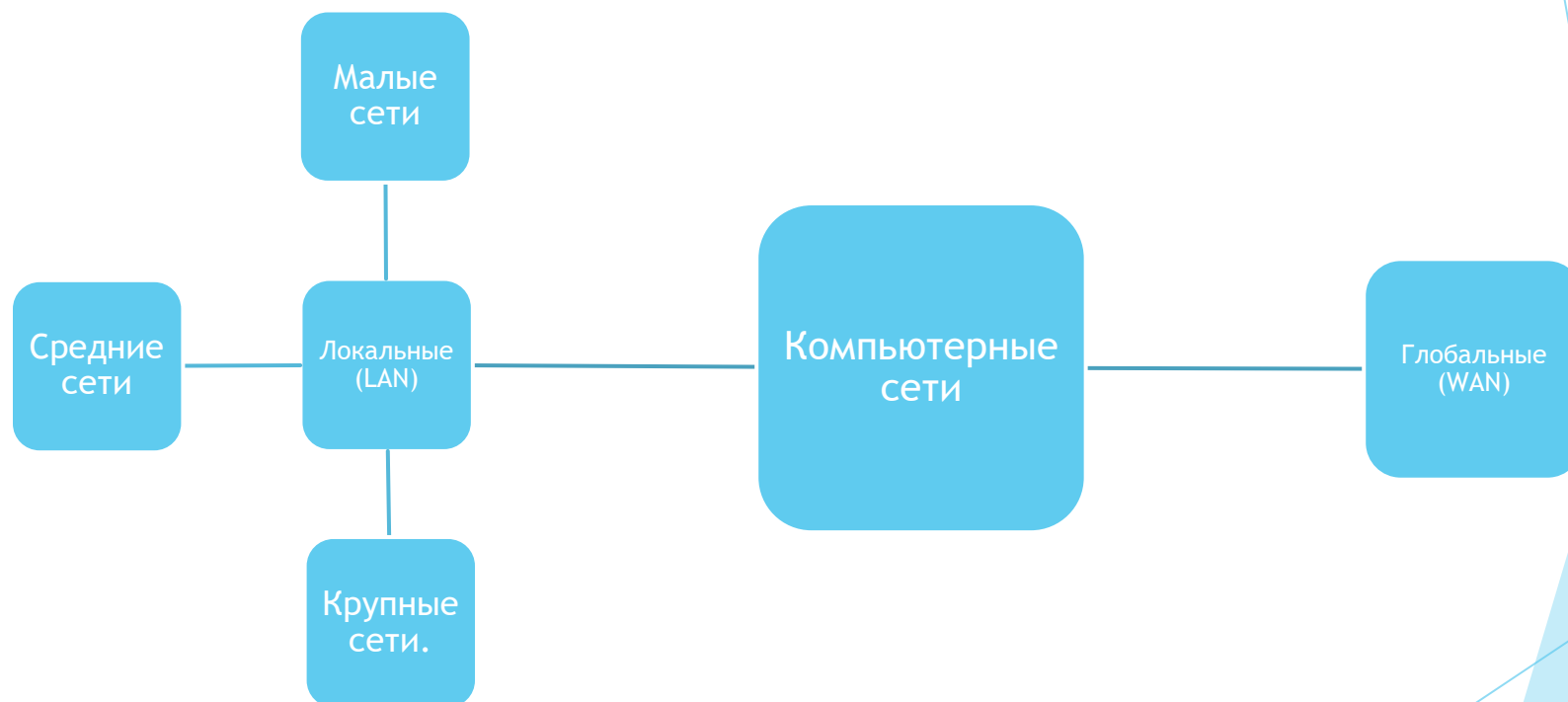
Введение в сетевое программирование

Сетевое программирование - процесс разработки приложений, которые способны обмениваться данными по компьютерным сетям.

Модель OSI



Компьютерные сети



Адресация в компьютерных сетях

Адресация в компьютерных сетях представляет собой систему идентификации устройств, обеспечивая их взаимодействие и передачу данных. Сетевые адреса позволяют определить отправителя и получателя информации в сети.

Виды адресов

Физический адрес (MAC-адрес) - уникальный идентификатор сетевого устройства в сети

Виды адресов

Логический адрес (IP - адрес, Internet Protocol Address) - это уникальный адрес устройства в сети, который позволяет маршрутизировать пакеты данных между разными узлами.

IP-адреса

Частные IP-адреса (серые) - зарезервированы для использования внутри частных сетей и не могут быть напрямую использованы в глобальной сети (в интернете)

Серые IP-адреса

RFC 1918:

- ▶ 10.0.0.0 — 10.255.255.255
- ▶ 172.16.0.0 — 172.31.255.255
- ▶ 192.168.0.0 — 192.168.255.255

Виды адресов

Порт (TCP/UDP) - это 16-битное число (диапазон от 0 до 65535), которое позволяет различать сервисы на одном устройстве. Например:

Модели сетевых приложений

- ▶ Клиент-сервера (Client-Server)
- ▶ Пиринговая сеть (P2P)
- ▶ Модель с использованием посредников (Middleware)
- ▶ Модель публикации/подписки (Pub/Sub)
- ▶ Модель Request-Response (обработка запросов)
- ▶ Реактивная модель (Reactive)

Windows sockets

Windows Sockets 2 (Winsock) позволяет программистам создавать расширенные приложения Интернета, интрасети и других сетевых приложений для передачи данных приложений по сети независимо от используемого сетевого протокола.

Использование Winsock

```
#define WIN32_LEAN_AND_MEAN
```

```
#include <iostream>
```

```
#include <windows.h>
```

```
#include <winsock2.h>
```

```
#include <ws2tcpip.h>
```

```
#include <iphlpapi.h>
```

```
#pragma comment(lib, "Ws2_32.lib")
```

```
int main() {  
    std::cout << "Hello World!\n";  
    return 0;  
}
```

Инициализация Winsock

```
WSADATA wsaData;
```

```
int iResult;
```

```
iResult = WSASStartup(MAKEWORD(2,2), &wsaData);
```

```
if (iResult != 0) {
```

```
    printf("WSASStartup failed: %d\n", iResult);
```

```
    return 1;
```

```
}
```

Клиентское приложение

Создание сокета для клиента

```
struct addrinfo *result = NULL,  
                *ptr = NULL,  
                hints;  
ZeroMemory( &hints, sizeof(hints) );  
hints.ai_family = AF_UNSPEC;  
hints.ai_socktype = SOCK_STREAM;  
hints.ai_protocol = IPPROTO_TCP;
```

Клиентское приложение

Создание сокета для клиента

```
#define DEFAULT_PORT "27015"
```

```
iResult = getaddrinfo(argv[1], DEFAULT_PORT, &hints, &result);
```

```
if (iResult != 0) {
```

```
    printf("getaddrinfo failed: %d\n", iResult);
```

```
    WSACleanup();
```

```
    return 1;
```

```
}
```


Клиентское приложение

Создание сокета для клиента

```
SOCKET ConnectSocket = INVALID_SOCKET;
```

```
ptr=result;
```

```
ConnectSocket = socket(ptr->ai_family, ptr->ai_socktype, ptr->ai_protocol);
```

```
if (ConnectSocket == INVALID_SOCKET) {
```

```
    printf("Error at socket(): %ld\n", WSAGetLastError());
```

```
    freeaddrinfo(result);
```

```
    WSACleanup();
```

```
    return 1;
```

```
}
```

Клиентское приложение

Подключение к сокету

```
iResult = connect( ConnectSocket, ptr->ai_addr, (int)ptr->ai_addrlen);  
if (iResult == SOCKET_ERROR) {  
    closesocket(ConnectSocket);  
    ConnectSocket = INVALID_SOCKET;  
}
```

Клиентское приложение

Отправка данных на клиенте

```
#define DEFAULT_BUFLen 512
const char *sendbuf = "this is a test";
int iResult;

iResult = send(ConnectSocket, sendbuf, (int) strlen(sendbuf), 0);
if (iResult == SOCKET_ERROR) {
    printf("send failed: %d\n", WSAGetLastError());
    closesocket(ConnectSocket); WSACleanup();
    return 1;
}
printf("Bytes Sent: %ld\n", iResult);
```

Клиентское приложение

Заккрытие соединения для отправки

```
iResult = shutdown(ConnectSocket, SD_SEND);  
if (iResult == SOCKET_ERROR) {  
    printf("shutdown failed: %d\n", WSAGetLastError());  
    closesocket(ConnectSocket);  
    WSACleanup();  
    return 1;  
}
```

Клиентское приложение

Получение данных от сервера

```
do {  
    iResult = recv(ConnectSocket, recvbuf, recvbuflen, 0);  
    if (iResult > 0)  
        printf("Bytes received: %d\n", iResult);  
    else if (iResult == 0)  
        printf("Connection closed\n");  
    else  
        printf("recv failed: %d\n", WSAGetLastError());  
} while (iResult > 0);
```

Клиентское приложение

Отключение клиента

```
iResult = shutdown(ConnectSocket, SD_SEND);  
if (iResult == SOCKET_ERROR) {  
    printf("shutdown failed: %d\n", WSAGetLastError());  
    closesocket(ConnectSocket);  
    WSACleanup();  
    return 1;  
}
```

```
closesocket(ConnectSocket);  
WSACleanup();  
return 0;
```

Серверное приложение

Создание сокета для сервера

```
#define DEFAULT_PORT "27015"

struct addrinfo *result = NULL, *ptr = NULL, hints;
ZeroMemory(&hints, sizeof (hints));
hints.ai_family = AF_INET;
hints.ai_socktype = SOCK_STREAM;
hints.ai_protocol = IPPROTO_TCP;
hints.ai_flags = AI_PASSIVE;

iResult = getaddrinfo(NULL, DEFAULT_PORT, &hints, &result);
if (iResult != 0) {
    printf("getaddrinfo failed: %d\n", iResult);
    WSACleanup();
    return 1;
}
```

Серверное приложение

Создание сокета для сервера

```
SOCKET ListenSocket = INVALID_SOCKET;
```

```
ListenSocket = socket(result->ai_family, result->ai_socktype, result->ai_protocol);
```

```
if (ListenSocket == INVALID_SOCKET) {  
    printf("Error at socket(): %ld\n", WSAGetLastError());  
    freeaddrinfo(result);  
    WSACleanup();  
    return 1;  
}
```


Серверное приложение

Привязка сокета

```
iResult = bind( ListenSocket, result->ai_addr, (int)result->ai_addrlen);  
if (iResult == SOCKET_ERROR) {  
    printf("bind failed with error: %d\n", WSAGetLastError());  
    freeaddrinfo(result);  
    closesocket(ListenSocket);  
    WSACleanup();  
    return 1;  
}  
  
freeaddrinfo(result);
```

Серверное приложение

Прослушивание сокета

```
if ( listen( ListenSocket, SOMAXCONN ) == SOCKET_ERROR ) {  
    printf( "Listen failed with error: %ld\n", WSAGetLastError() );  
    closesocket(ListenSocket);  
    WSACleanup();  
    return 1;  
}
```

Серверное приложение

Принятие подключения

```
SOCKET ClientSocket;
```

```
ClientSocket = INVALID_SOCKET;
```

```
ClientSocket = accept(ListenSocket, NULL, NULL);
```

```
if (ClientSocket == INVALID_SOCKET) {
```

```
    printf("accept failed: %d\n", WSAGetLastError());
```

```
    closesocket(ListenSocket);
```

```
    WSACleanup();
```

```
    return 1;
```

```
}
```

```
closesocket(ListenSocket);
```

Клиентское приложение

Получение и отправка данных в сокете

```
#define DEFAULT_BUFLen 512
char recvbuf[DEFAULT_BUFLen];
int iResult, iSendResult;
int recvbuflen = DEFAULT_BUFLen;
do {
    iResult = recv(ClientSocket, recvbuf, recvbuflen, 0);
    if (iResult > 0) {
        printf("Bytes received: %d\n", iResult);
        iSendResult = send(ClientSocket, recvbuf, iResult, 0);
        if (iSendResult == SOCKET_ERROR) {
            printf("send failed: %d\n", WSAGetLastError());
            closesocket(ClientSocket);
            WSACleanup();
            return 1;
        }
        printf("Bytes sent: %d\n", iSendResult);
    }
    else if (iResult == 0)
        printf("Connection closing...\n");
    else {
        printf("recv failed: %d\n", WSAGetLastError());
        closesocket(ClientSocket);
        WSACleanup();
        return 1;
    }
} while (iResult > 0);
```

Серверное приложение

Отключение сервера

```
iResult = shutdown(ClientSocket, SD_SEND);  
if (iResult == SOCKET_ERROR) {  
    printf("shutdown failed: %d\n", WSAGetLastError());  
    closesocket(ClientSocket);  
    WSACleanup();  
    return 1;  
}  
  
closesocket(ClientSocket);  
WSACleanup();  
return 0;
```